Mathematical Derivations and Code Mappings for BrainKAN Interpretability Benchmark

Xin Qi

October 2026

Abstract

This document outlines the formal mathematical framework for the BrainKAN explainability project, providing one-to-one mappings between the theoretical derivations and their concrete implementations in the Python codebase. Crucially, it formulates rigorous statistical null-models (permutation testing) to demonstrate that many apparent non-linear computational effects in macro-scale connectomics are artifacts of input distribution shifts, highlighting the necessity of structural null-calibration.

1 Edge-Specific KAN Convolution Architecture

1.1 Mathematical Formulation

Let the task-fMRI functional brain network be represented as a directed graph $G = (V, E)$, where V represents the brain regions of interest (ROIs) and E represents the edges (functional connectivities). For a given layer l, the node features are updated via:

$$h_j^{(l+1)} = \sum_{i \in \mathcal{N}(j)} \Phi_{ij}^{(l)} \left(h_i^{(l)} \right) \tag{1}$$

Unlike conventional message-passing layers with shared transformations, BrainKAN parameterizes Φ_{ij} uniquely for every existing edge $(i, j) \in E$ using B-splines.

For a scalar input x, the edge-specific function is defined as:

$$\Phi_{ij}(x) = W_{ij}^{base} \, \mathrm{SiLU}(x) + \mathrm{spline}_{ij}(x) = W_{ij}^{base} \, \mathrm{SiLU}(x) + \sum_{k=1}^{K} w_{ij,k} B_k(x) \tag{2}$$

where $B_k(x)$ are the B-spline basis functions defined recursively over a uniform grid, and $w_{ij,k}$ are the learnable spline coefficients specifically allocated for the edge (i, j).

The updated node features are:

$$h_i' = R(h_i) + \sum_j A_{ij} \Phi_{ij}(h_j) \tag{3}$$

where R is an optional residual connection.

1.2 Code Implementation

File: `models/kan.py`
Class/Function: `EdgeSpecificKANConv.forward()` and `_b_spline_basis()`
The recursive B-spline basis evaluation is centrally implemented in `_b_spline_basis()`, which is imported throughout the repository to guarantee numerical consistency (e.g., in `02_q3b_common_reference_geometry.py`).

2 Q1: Function Characterization and Deviation-from-Linearity (DFL)

2.1 Mathematical Formulation

To characterize whether the learned edge function Φ_{ij} is actually nonlinear, we evaluate it over a continuous domain $T = [t_{min}, t_{max}]$ and compute the Deviation-From-Linearity (DFL). Let $y(t) = \Phi_{ij}(t)$. We fit a simple linear regression $\hat{y}(t) = at + b$. The DFL is defined as $1 - R^2$, where:

$$R^2 = 1 - \frac{\sum_{t\in T}(y(t) - \hat{y}(t))^2}{\sum_{t\in T}(y(t) - \bar{y})^2} \tag{4}$$

$$\text{DFL} = 1 - R^2 = \frac{\sum_{t\in T}(y(t) - \hat{y}(t))^2}{\sum_{t\in T}(y(t) - \bar{y})^2} \tag{5}$$

A higher DFL indicates that the edge function deviates significantly from a pure linear transformation.

2.2 Code Implementation

File: `01_q1_q2_q3a_naive_analysis.py`
Function: `compute_function_signatures()`
The code computes `ss_res` (residual sum of squares) and `ss_tot` (total sum of squares) to derive R^2, and then outputs $\text{DFL}_e = 1 - R^2$. The edge-level DFL is averaged across output channels: $NL_e = \frac{1}{16}\sum_o NL_{e,o}$.

3 Q2: Function Taxonomy

3.1 Mathematical Formulation

To determine if edge functions group into a discrete taxonomy of biological computations, we standardize the function signatures and apply Ward hierarchical clustering for $k = 2, \ldots, 7$. Cluster quality and fold-level cross-validation stability are measured using Silhouette and Adjusted Rand Index (ARI):

$$\text{Silhouette} \approx 0.18 - 0.19, \quad \text{ARI} \approx 0 \tag{6}$$

This indicates no reproducible small-k discrete taxonomy is detected across folds.

3.2 Code Implementation

Files: `01_q1_q2_q3a_naive_analysis.py` (Real) and `05_q2_null_taxonomy.py` (Null).

4 Q3A: Naive Curvature Remodeling

4.1 Mathematical Formulation

The naive curvature effect (ΔC) assesses whether the second derivative of the edge function changes across task conditions ($c \in \{0BK, 2BK\}$). For an edge e, the condition-specific naive curvature is calculated over the subject-specific input distribution $X^{(c)}$:

$$C_e(x) = \frac{1}{16}\sum_{o=1}^{16} |f''_{e,o}(x)| \tag{7}$$

The shift in curvature is then:

$$\Delta C_e = C_e(x_j^{2BK}) - C_e(x_j^{0BK}) \tag{8}$$

Note: As discussed, ΔC_e is mathematically confounded by kinematic shifts. Locally, away from points where $f''(x) = 0$, a first-order expansion illustrates how operating-regime shifts can induce apparent changes in curvature statistics:

$$\Delta C \approx C'(x_0) \cdot \Delta x + \frac{1}{2}C''(x_0) \cdot \Delta x^2 + \cdots \tag{9}$$

This demonstrates why $\Delta C^{real} \approx \Delta C^{null}$ under a label-shuffled null, representing an identifiability failure of the naive estimand.

4.2 Code Implementation

File: `01_q1_q2_q3a_naive_analysis.py`
The script evaluates the second derivative using PyTorch's `torch.autograd.grad` on the actual task-specific inputs.

5 Q3B: Common-Reference Null Calibration

5.1 Mathematical Formulation

To eliminate the input-regime confounder, we evaluate the learned geometry over a fixed, condition-independent reference grid x_j^{ref}, constructed via quantiles of the pooled training distribution:

$$x_j^{ref} = Q_{\text{pooled training},j}(q) \tag{10}$$

The intrinsic structural nonlinearity (using DFL as the test statistic G_e) is computed for the real model (G^{real}) and compared against $R = 100$ null models (G^{null}):

$$G_e = \frac{1}{16} \sum_o (1 - R^2_{e,o,\text{ref}}) \tag{11}$$

The observed statistic is:

$$T_e^{real} = G_e^{real} \tag{12}$$

and compared against $\{G_e^{null,r}\}_{r=1}^R$. The effect size measuring excess geometry is:

$$\Psi_e = G_e^{real} - E[G_e^{null}] \tag{13}$$

Ψ_e represents the excess structural variance relative to the empirical null expectation. Note that while this calibrates for baseline model flexibility, it does not guarantee isolation of purely "true biological signal," as other unmodelled confounders (e.g., specific preprocessing artifacts or cohort-level structure) may still be present. Statistical significance (yielding 0/240 FDR) is determined by whether T_e^{real} lies in the extreme tail of the empirical null distribution.

5.2 Code Implementation

File: `02_q3b_common_reference_geometry.py`
Models are trained identically under true labels vs. subject-level swapped labels. All functions are probed strictly on $X_{ref} = [-8, 12]$.

6 Q3C: Held-Out Calibrated Effective Gain

6.1 Mathematical Formulation

Because Q3B found no excess structural nonlinearity, we test for simpler computational changes: changes in the Effective Gain (the first derivative). To ensure unbiased estimation, we use a fixed condition-independent reference grid X_{ref}, strictly computed from the *training fold* to prevent data leakage.

$$G_{e,c} = \frac{1}{16} \sum_o \mathbb{E}_{x \sim P_c} \left[|f'_{e,o}(x)| \right] \tag{14}$$

Since $f'(x)$ is evaluated via numerical differentiation:

$$f'(x) \approx \frac{f(x+\epsilon) - f(x-\epsilon)}{2\epsilon} \tag{15}$$

The paired difference in effective gain for a subject is:

$$dG_i = G_i^{2BK} - G_i^{0BK} \tag{16}$$

We test the one-sided hypothesis that the paired differences from the real model exhibit strictly greater effective structural gain than the null:

$$H_0 : \mu_{\Delta G} \leq 0 \quad \text{vs} \quad H_1 : \mu_{\Delta G} > 0 \tag{17}$$

Let $T_e^{real} = \sum_i (dG_i^{real} - dG_i^{null})$ and for each permutation r, the null statistic is $T_e^{null,r} = \sum_i s_i^{(r)} (dG_i^{real} - dG_i^{null})$ with $s_i^{(r)} \in \{-1, +1\}$. The permutation p-value with R sign-flips is:

$$p_e = \frac{1 + \sum_r I(T_e^{null,r} \geq T_e^{real})}{R + 1} \tag{18}$$

(If $T_e^{real} \leq 0$, we trivially set $p_e = 1.0$)

$$d_z = \frac{\mu_{\Delta G}}{\sigma_{\Delta G}} \tag{19}$$

where $\mu_{\Delta G}$ and $\sigma_{\Delta G}$ are the mean and standard deviation of $dG_{e,s}^{real} - dG_{e,s}^{null}$ across all test subjects.

6.2 Code Implementation

File: `04_q3c_effective_gain.py`
Functions: `evaluate_Q3C()` and the subsequent statistical testing block.
Cohen's d_z is explicitly calculated as `observed_mean / np.std(diff)` and saved in `q3c_final_table.csv`.

7 Synthetic Identifiability Benchmark

7.1 Mathematical Formulation

To define the identifiability boundary, we compare the measured effective gain ΔG across synthetic scenarios: baseline (S_0), input shift (S_1), and gain modulation (S_2).

$$S_1(\text{input shift}) : \Delta G = -0.087 \pm 0.13 \tag{20}$$

$$S_2(\text{gain modulation}) : \Delta G = -0.082 \pm 0.13 \tag{21}$$

$$S_1 \approx S_2 \tag{22}$$

This demonstrates empirically that input shift and gain modulation are confusable under the tested Q3C estimator. To ensure the estimator can detect true structural remodeling when present, S_5 tests a strong true structural change via a common-support comparison distance D:

$$D_{S5} = 3.574 \pm 0.75 \gg D_{S0} = 0.153 \pm 0.25 \tag{23}$$

Rejecting $H_0 : S_5 = S_0$ proves the estimator responds to the tested remodeling regime.

7.2 Code Implementation

File: `03_synthetic_identifiability_benchmark.py`
Simulates S0-S5 using synthetic 1D splines.

8 Cross-Cohort Distribution Shift and Null-Calibrated Identifiability

8.1 Mathematical Formulation

When transferring a nonlinear representation learned on a source cohort (HCP) to a target cohort (CHCP) with substantial distribution compression, naive interpretation is confounded by the change in reference domain.

First, to characterize the magnitude of the global distribution shift before transferring models, we calculate the shift in mean $\Delta\mu$ and the variance ratio ν between the target (CHCP) and source (HCP) cohorts across all input features:

$$\Delta\mu = \mu_{CHCP} - \mu_{HCP} \tag{24}$$

$$\nu = \frac{\mathrm{Var}_{CHCP}}{\mathrm{Var}_{HCP}} \tag{25}$$

The substantial compression in variance ($\nu \approx 0.1412$) constitutes a non-trivial distributional shift that can severely distort geometry.

Second, we define the *Support Overlap* O_j to quantify the potential extrapolation risk when applying a frozen reference grid X_j^{ref} (derived from HCP) to the empirical support of the target CHCP data X_j^{CHCP}:

$$O_j = \frac{1}{99}\sum_{k=1}^{99} \mathbb{I}\left(x_{j,k}^{ref} \in [q_1(X_j^{CHCP}),\, q_{99}(X_j^{CHCP})]\right) \tag{26}$$

where $x_{j,k}^{ref}$ are the 99 quantiles of the reference distribution, and q_1, q_{99} are the extreme percentiles defining the empirical support of the target data.

Third, we construct the *2×2 Identifiability Matrix* to decouple true nonlinear variation from reference-domain dependence. We evaluate the non-linearity G under four conditions:

$$M = \begin{bmatrix} G_{Native}^{real} & G_{Frozen}^{real} \\ G_{Native}^{null} & G_{Frozen}^{null} \end{bmatrix} \tag{27}$$

A synchronous inflation in both the Real and Null models ($G_{Frozen} > G_{Native}$) indicates that the apparent geometric change is primarily an artifact of the reference shift rather than biological signal.

Finally, to isolate task-associated variation from model-induced and reference-dependent geometric deformation, we calibrate the raw geometry G_e^{real} using an empirical null distribution derived from R independent label-shuffled models evaluated on the same fixed reference domain. The *Standardized Separation* (Z-score) is defined as:

$$Z_e = \frac{G_e^{real} - \mu(G_e^{null})}{\sigma(G_e^{null})} \tag{28}$$

where $\mu(G_e^{null})$ and $\sigma(G_e^{null})$ are the sample mean and standard deviation of the structural nonlinearity test statistic G_e computed across the R null models.

8.2 Code Implementation

Files:

- `experiments/chcp_external_validation.py`
- `experiments/chcp_support_overlap.py`
- `experiments/print_chcp_2x2.py`

The standardized separation calculation directly maps the formulation by extracting the robust empirical mean and standard deviation from the $R = 100$ null permutations, providing a highly stable metric against cross-cohort shift.